

Document Loaders in LangChain

1. Simple & Clear Notes

◆ Recap

- So far in the playlist:
 - Covered **LangChain Components** (Models, Prompts, Chains).
 - Covered **Core Concepts** (like Runnables in depth).
 - Now starting **RAG (Retrieval-Augmented Generation)** based applications.
-

◆ What is RAG?

- RAG = **Information Retrieval + Language Generation**.
- Helps LLMs when:
 - Data is not in training (e.g., current events).
 - Data is private (personal emails, company docs).
 - Data is too large (big PDFs beyond context window).
- Benefits:
 - Up-to-date answers.
 - Privacy (don't upload to ChatGPT).
 - Handles large documents (split into chunks).

👉 RAG apps need 4 key components:

1. **Document Loaders**
 2. **Text Splitters**
 3. **Vector DBs**
 4. **Retrievers**
-

◆ Document Loaders – Definition

- **Purpose:** Load data from different sources into a **standardized format** = Document object.
 - **Why needed?** Data exists in many places (PDFs, text files, databases, cloud, web pages).
 - **Standard Format:**
 - `page_content`: actual text
 - `metadata`: info like source, author, creation date
-

◆ Most Common Document Loaders

1. Text Loader

- Loads .txt files into Document objects.
- Example use cases: logs, transcripts, code files.

Code:

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("cricket.txt", encoding="utf-8")
docs = loader.load()

print(docs[0].page_content)    # content
print(docs[0].metadata)        # metadata
```

2. PyPDF Loader

- Loads **PDFs page by page**.
- Each page → separate Document.
- Uses pypdf internally.
- Good for text PDFs, not for scanned/complex layouts.

Code:

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("deep_learning_curriculum.pdf")
docs = loader.load()

print(len(docs))          # number of pages
print(docs[0].page_content) # first page
print(docs[0].metadata)    # metadata incl. page number
```

- Other PDF loaders:
 - pdfplumber (for tables).
 - UnstructuredPDFLoader / AmazonTextractPDFLoader (for scanned PDFs).
 - PyMuPDFLoader (for layouts).

3. Directory Loader

- Loads **multiple files** from a folder.
- Supports glob patterns (*.pdf, *.txt, etc.).

Code:

```
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader("books", glob="*.pdf", loader_cls=PyPDFLoader)
docs = loader.load()

print(len(docs)) # total docs = pages across all PDFs
```

- Issue: load() = eager loading → loads everything in memory.
- Solution: lazy_load() = generator → loads one doc at a time (saves memory).

4. Web Base Loader

- Loads content from web pages (HTML → text).
- Internally uses:
 - requests (fetch HTML).
 - BeautifulSoup (extract text).
- Good for static sites, blogs, news articles.
- Not good for heavy JS pages → use SeleniumURLLoader instead.

Code:

```
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader("https://example.com/page")
docs = loader.load()

print(docs[0].page_content)
```

- Can pass **list of URLs** too.

5. CSV Loader

- Each **row** in CSV → separate Document.

Code:

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("social_network_ads.csv")
docs = loader.load()

print(len(docs))          # rows in CSV
print(docs[0].page_content)
print(docs[0].metadata)
```

- Works well for data analysis questions with LLMs.

◆ Other Document Loaders

- Categories in LangChain Docs:
 - Web (e.g., Sitemap loader, Hyperbrowser).
 - PDFs (many variations).
 - Cloud (S3, Azure, GDrive, Dropbox).
 - Social Media (Slack, Notion, etc.).
 - Messaging / Productivity Tools.
 - Common file types (JSON, CSV, YouTube transcripts).

👉 Learn only what you need for your project.

◆ Custom Document Loader

- If no loader exists → create your own.
- Inherit from BaseLoader.
- Implement `.load()` / `.lazy_load()` with your logic.

◆ Key Takeaways

- Document Loaders = **first step in RAG apps**.
 - They convert any data source → standard Document format.
 - Four most used loaders: **Text, PDF, Web, CSV**.
 - DirectoryLoader & `lazy_load()` help with multiple/large files.
 - You can build custom loaders when needed.
-